

TITRE PROFESSIONNEL DÉVELOPPEUR WEB ET WEB MOBILE

ModHub

Plateforme communautaire de partage de mods pour jeux vidéo

Dossier de projet

[Prénom NOM du candidat]

[Nom du centre de formation]

Session [date de la session d'examen]

Sommaire

1. Présentation du candidat à compléter
2. Présentation du projet
 - a. Contexte et origine
 - b. Objectifs et public visé
 - c. Fonctionnalités
3. Gestion de projet
4. Architecture et choix techniques
 - a. Vue d'ensemble
 - b. Modèle de données
 - c. Justification des choix
5. Compétences — Bloc 1 : Front-end
 - a. Maquetter les interfaces
 - b. Réaliser les interfaces statiques
 - c. Développer la partie dynamique
6. Compétences — Bloc 2 : Back-end
 - a. Base de données relationnelle
 - b. Accès aux données SQL et NoSQL
 - c. Composants métier serveur
7. Sécurisation de l'application
8. Tests
9. Bilan et perspectives
10. Annexes

1. Présentation du candidat

À compléter par le candidat. Cette section doit rester personnelle : parcours, formation suivie, expériences précédentes, motivation pour le développement web, ce qui a conduit au choix de ce projet précis. Un dossier crédible aux yeux du jury est un dossier que le candidat peut défendre avec ses propres mots.

[Parcours professionnel / scolaire avant la formation]

[Nom de la formation, organisme, dates]

[Motivations personnelles pour ce métier et pour ce projet en particulier — le lien avec le modding de jeux vidéo, une pratique déjà expérimentée par le candidat, est un bon axe à développer]

2. Présentation du projet

2.1 Contexte et origine

ModHub est une plateforme communautaire permettant à des créateurs de mods pour jeux vidéo de publier leurs créations (avec gestion de versions et changelog), et à la communauté de les découvrir, les noter, les commenter et suivre leurs mises à jour. Le projet part d'un constat simple : les communautés de modding s'organisent souvent autour d'outils génériques (forums, Discord, plateformes tierces) mal adaptés au suivi structuré d'un mod dans la durée — versions, compatibilité, retours qualitatifs de la communauté.

[À personnaliser : décrire ici l'expérience concrète du candidat dans le modding — par exemple le travail réalisé sur des mods Arma Reforger — qui légitime le choix de ce sujet]

2.2 Objectifs et public visé

Public	Besoin
Créateurs de mods	Publier un mod, gérer ses versions, suivre son adoption (téléchargements, notes)
Joueurs / communauté	Trouver rapidement des mods pertinents, évaluer leur qualité avant de les installer, suivre les mods qui les intéressent
Modérateurs	Garder un catalogue de qualité en validant les publications avant diffusion publique

2.3 Fonctionnalités

- Catalogue public avec recherche et filtres (par jeu, catégorie)
- Fiche mod détaillée : description, versions/changelog, métadonnées spécifiques au jeu, notes, commentaires
- Comptes utilisateurs avec rôles (utilisateur / administrateur)
- Espace créateur : publication de mods, ajout de versions
- Notation et commentaires communautaires
- Suivi de mods (follow) et fil d'activité personnalisé
- Modération administrateur (validation / rejet des publications)

3. Gestion de projet

3.1 Méthodologie

Le développement a suivi une démarche en phases successives, inspirée des principes agiles adaptés à un projet solo : cadrage du besoin, choix d'architecture documenté avant l'implémentation, découpage en tranches verticales livrables (d'abord l'authentification de bout en bout, puis le cœur métier des mods, puis le front-end, puis le durcissement sécurité), avec vérification systématique à chaque étape plutôt qu'une phase de test globale en fin de projet.

3.2 Outils

Besoin	Outil
Gestion de version	Git, dépôt hébergé sur GitHub
Hébergement base relationnelle (développement)	Supabase (PostgreSQL managé)
Base NoSQL (développement)	MongoDB local
Tests	Vitest + Supertest (tests d'intégration)
Suivi de tâches	[à compléter si un outil type Trello/Notion a été utilisé]

3.3 Planning

[À compléter : dates réelles de démarrage/livraison, éventuellement un diagramme de Gantt simplifié si le centre de formation l'exige]

4. Architecture et choix techniques

4.1 Vue d'ensemble

L'application est structurée en deux projets distincts communiquant via une API REST :

```
ModHub/
├── frontend/    React 19 + TypeScript (Vite) - interface utilisateur
├── backend/     Node.js + Express 5 + TypeScript - API REST
│   ├── PostgreSQL (Prisma 7) → données relationnelles
│   └── MongoDB (Mongoose)    → fil d'activité, métadonnées par jeu
└── README.md
```

```

Mod (id, title, slug UNIQUE, summary, description, gameKey, status, authorId → User, categoryId
Version (id, modId → Mod, versionLabel, changelog, fileUrl, fileSizeBytes, downloadCount)
    UNIQUE(modId, versionLabel)
Rating (id, modId → Mod, userId → User, value)                UNIQUE(modId, userId)
Comment (id, modId → Mod, userId → User, body, createdAt)
Follow (id, modId → Mod, userId → User)                        UNIQUE(modId, userId)

```

4.2.2 MongoDB — collections

Collection	Usage	Pourquoi NoSQL plutôt que relationnel
ActivityEvent	Fil d'activité (nouveau mod, nouvelle version, nouveau commentaire)	La forme du document varie selon le type d'événement (<code>payload</code> différent pour chaque <code>type</code>) ; écriture fréquente, pas de besoin de jointure
ModMetadata	Champs spécifiques à chaque jeu (ex. identifiant Workshop et DLC requis pour un mod Arma Reforger, version du moteur et loader pour un mod Minecraft)	Schéma volontairement variable selon <code>gameKey</code> — le modéliser en relationnel imposerait un anti-pattern de type EAV (Entity-Attribute-Value)

4.3 Justification des choix technologiques

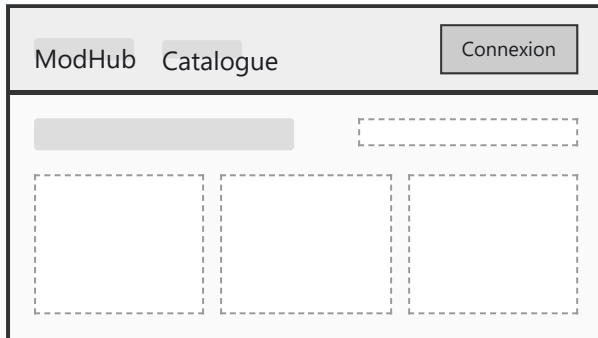
Choix	Justification
React + TypeScript	Typage statique de bout en bout (partagé conceptuellement entre les types d'API et les composants), écosystème très documenté, adapté à une interface fortement interactive
Express 5	Minimaliste, laisse le contrôle total sur l'architecture (pas de convention imposée type NestJS), gestion native des erreurs asynchrones (une exception levée dans un contrôleur <code>async</code> est automatiquement transmise au middleware d'erreur, sans bloc try/catch répétitif)
Prisma	Requêtes typées de bout en bout, migrations versionnées et rejouables, lisibilité du schéma comme documentation vivante de la base
PostgreSQL (Supabase)	Base relationnelle robuste, hébergement managé gratuit qui évite une dépendance à Docker/WSL2 indisponible sur la machine de développement
MongoDB	Adapté aux deux cas d'usage identifiés (schéma variable, écriture d'événements), bibliothèque Mongoose mature pour la validation de schéma côté application
JWT + bcryptjs	Authentification sans état côté serveur (scalable horizontalement), hachage de mot de passe avec coût réglable ; bcryptjs plutôt que bcrypt pour éviter une dépendance à compilation native

5. Compétences — Bloc 1 : développer la partie front-end

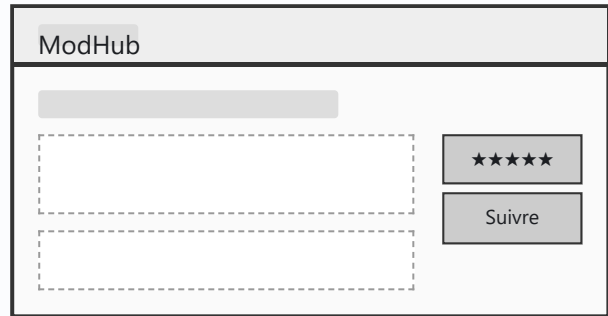
Compétence

5.1 Maquetter des interfaces utilisateur web ou web mobile

Avant l'implémentation, les écrans principaux ont été maquetés en basse fidélité pour valider la structure de l'information (hiérarchie visuelle, zones d'action, parcours) indépendamment du rendu graphique final.



Maquette basse fidélité — Catalogue



Maquette basse fidélité — Fiche mod

Ces maquettes ont été formalisées à un niveau volontairement schématique (boîtes grisées, pas de contenu réel) pour se concentrer sur la disposition avant de passer à l'intégration. Pour l'entretien technique, il est recommandé de reproduire ces maquettes dans un outil dédié (Figma) et d'expliquer les itérations éventuelles entre la maquette et le résultat final.

Compétence

5.2 Réaliser des interfaces utilisateur statiques

L'habillage visuel repose sur une feuille de style unique (`index.css`) centralisant les jetons de couleur (variables CSS), sans framework CSS externe — choix assumé pour garder un contrôle total sur le rendu et éviter une dépendance supplémentaire sur un projet de cette taille.

```
:root {
  --color-bg: #0f1115;
  --color-surface: #171a21;
  --color-accent: #5b8def;
  --color-danger: #e0554f;
  --color-success: #3fb27f;
}
```

La mise en page utilise Flexbox et CSS Grid (`grid-template-columns: repeat(auto-fill, minmax(260px, 1fr))`) pour la grille de mods, ce qui rend le catalogue responsive sans media query dédiée pour les tailles intermédiaires.

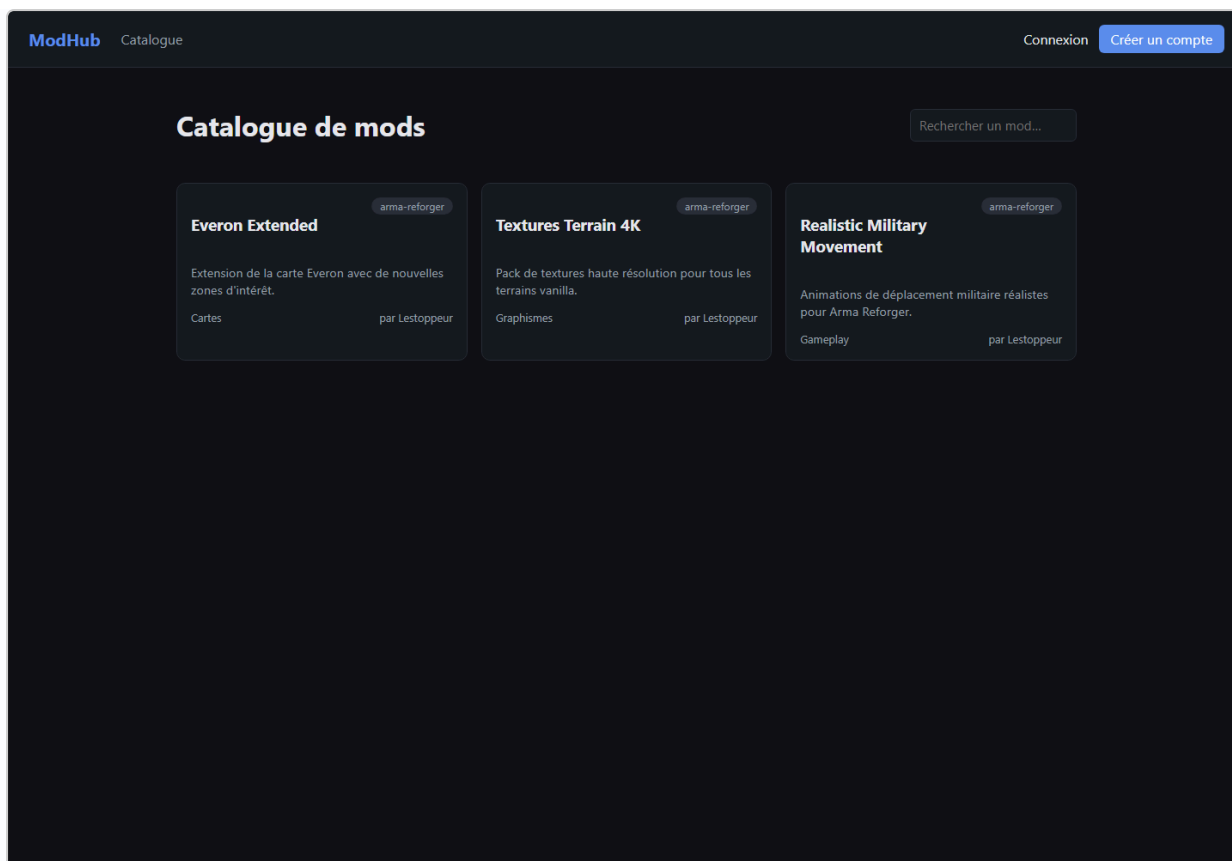


Fig. 1 — Catalogue public, rendu réel de l'application (capture d'écran)

Compétence

5.3 Développer la partie dynamique des interfaces utilisateur

Plusieurs mécanismes illustrent la partie dynamique de l'application :

- **Recherche en temps réel avec debounce** — chaque frappe déclenche une nouvelle requête après 300 ms d'inactivité, évitant de saturer l'API à chaque caractère tapé :

```
useEffect(() => {
  const handle = setTimeout(() => {
    setLoading(true);
    modsApi.search({ q: query || undefined })
      .then((res) => setItems(res.items))
      .finally(() => setLoading(false));
  }, 300);
  return () => clearTimeout(handle);
}, [query]);
```

- **Contexte d'authentification global** (React Context + hook `useAuth`) qui persiste la session en `localStorage` et pilote l'affichage conditionnel de la navigation (liens "Publier un mod", "Modération" visibles seulement selon l'état de connexion et le rôle).

- **Routes protégées** (`ProtectedRoute`) redirigeant vers la page de connexion si l'utilisateur n'est pas authentifié, ou vers l'accueil si son rôle ne correspond pas à la page demandée (ex. un utilisateur non admin ne peut pas accéder à `/admin`).
- **Formulaires contrôlés** avec retour d'erreur asynchrone affiché sans rechargement de page (inscription, connexion, publication de mod, notation, commentaire).

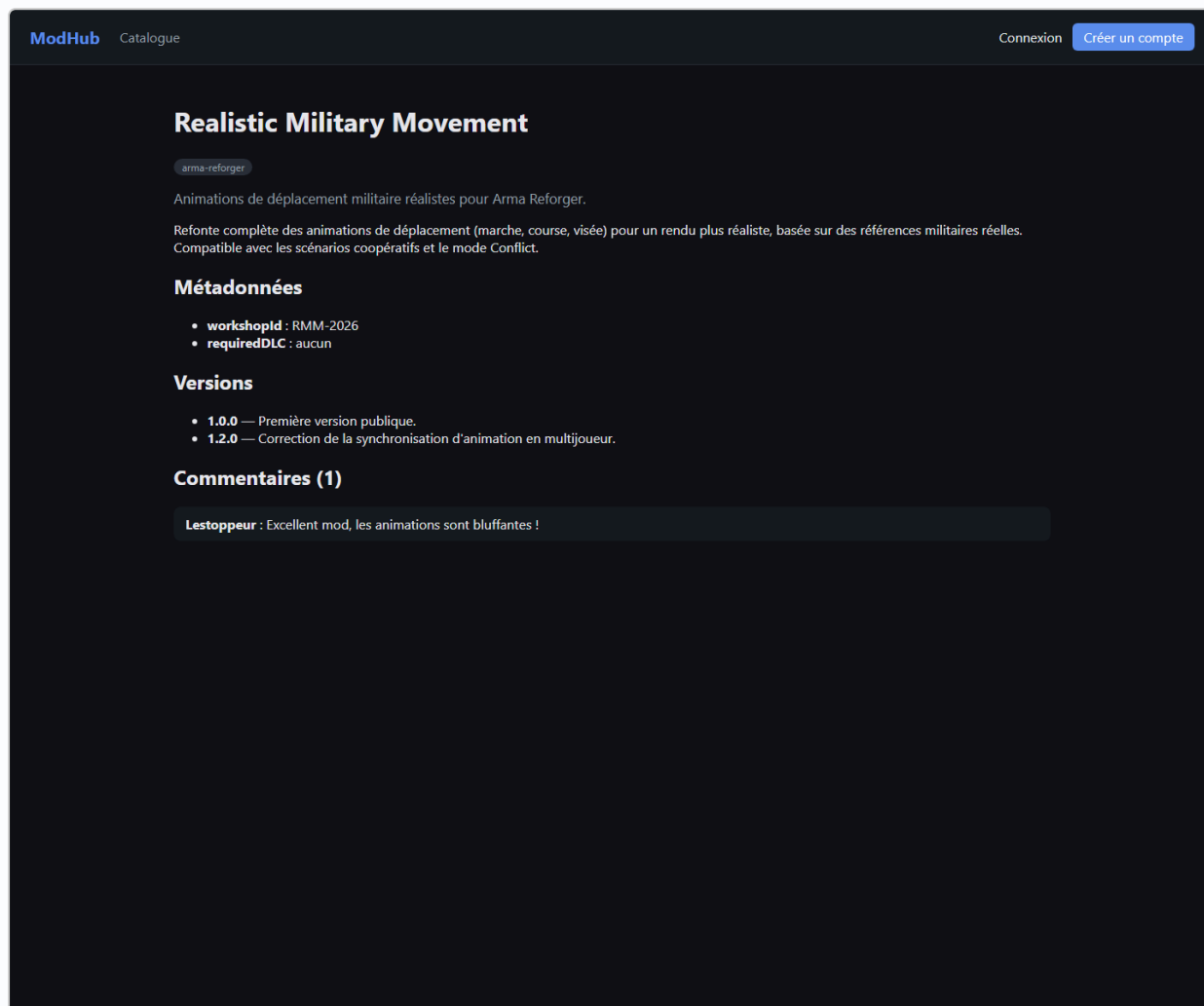


Fig. 2 — Fiche mod : métadonnées, versions et commentaires assemblés dynamiquement depuis deux sources de données différentes (PostgreSQL + MongoDB)

6. Compétences — Bloc 2 : développer la partie back-end

Compétence

6.1 Mettre en place une base de données relationnelle

Le schéma PostgreSQL est défini et versionné via Prisma (`prisma/schema.prisma`), avec migrations SQL générées et historisées dans `prisma/migrations/`. Extrait :

```
model Mod {
  id          String    @id @default(uuid())
  title       String
  slug        String    @unique
  status      ModStatus @default(PENDING)
  authorId    String
  author      User       @relation(fields: [authorId], references: [id], onDelete: Cascade)
  categoryId  String
  category    Category   @relation(fields: [categoryId], references: [id])
  versions    Version[]
  ratings     Rating[]
  @@index([gameKey])
  @@index([status])
}
```

Les contraintes d'unicité composites (`@@unique([modId, userId])` sur `Rating` et `Follow`) garantissent au niveau base de données qu'un utilisateur ne peut noter ou suivre un mod qu'une seule fois, sans dépendre uniquement d'une vérification applicative.

Compétence

6.2 Développer des composants d'accès aux données SQL et NoSQL

Deux jeux de repositories cohabitent, chacun encapsulant sa technologie :

```
// repositories/sql/mod.repository.ts - Prisma / PostgreSQL
search(filters, skip, take) {
  const where: Prisma.ModWhereInput = {
    status: filters.status ?? 'APPROVED',
    ...(filters.search && { OR: [
      { title: { contains: filters.search, mode: 'insensitive' } },
      { summary: { contains: filters.search, mode: 'insensitive' } },
    ]}),
  };
  return prisma.$transaction([
    prisma.mod.findMany({ where, skip, take, orderBy: { createdAt: 'desc' } }),
    prisma.mod.count({ where }),
  ])
```

```
});
}
```

```
// repositories/nosql/activity.repository.ts - Mongoose / MongoDB
setMetadata(modId, gameKey, fields) {
  return ModMetadataModel.findOneAndUpdate(
    { modId },
    { modId, gameKey, fields },
    { upsert: true, returnDocument: 'after' },
  );
}
```

Le service métier orchestre les deux sans les mélanger : la création d'un mod écrit l'entité principale en PostgreSQL puis, si des métadonnées spécifiques au jeu sont fournies, les stocke séparément en MongoDB et journalise l'événement dans le fil d'activité :

```
async createMod(authorId, input) {
  const mod = await modRepository.create({ ...input, author: { connect: { id: authorId } } });
  if (input.gameMetadata) {
    await activityRepository.setMetadata(mod.id, input.gameKey, input.gameMetadata);
  }
  await activityRepository.record('NEW_MOD', mod.id, authorId, { title: mod.title });
  return mod;
}
```

Ce comportement est couvert par un test d'intégration réel (`tests/mods.test.ts`) qui crée un mod via l'API, puis vérifie directement en base que le document MongoDB correspondant existe avec les bons champs — la preuve que les deux bases sont réellement utilisées, pas seulement déclarées dans le schéma.

Compétence

6.3 Développer des composants métier côté serveur

La couche `services/` concentre les règles métier, par exemple le contrôle de propriété avant l'ajout d'une version :

```
async addVersion(modId, authorId, input) {
  const mod = await modRepository.findById(modId);
  if (!mod) throw new AppError(404, 'Mod introuvable');
  if (mod.authorId !== authorId) {
    throw new AppError(403, "Seul l'auteur du mod peut publier une version");
  }
  const version = await modRepository.addVersion(modId, input);
  await activityRepository.record('NEW_VERSION', modId, authorId, { versionLabel: version.

```

```
    return version;  
}
```

Autres composants métier notables : calcul de moyenne de notes via agrégation Prisma (`prisma.rating.aggregate`), génération de slugs uniques avec normalisation Unicode (suppression des accents), workflow de modération à trois états (`PENDING` → `APPROVED` / `REJECTED`) contrôlé par le rôle de l'utilisateur.



7. Sécurisation de l'application

Risque (OWASP)	Mesure appliquée
Injection SQL	Aucune requête SQL brute concaténée ; toutes les requêtes passent par l'API typée de Prisma (requêtes paramétrées par construction)
Injection NoSQL	Les requêtes Mongoose utilisent l'API objet du driver, jamais de construction de requête à partir d'une chaîne fournie par l'utilisateur
Mots de passe	Hachage bcryptjs (12 rounds), jamais stocké ni renvoyé en clair ; le message d'erreur de connexion est identique pour "email inconnu" et "mot de passe incorrect" (évite l'énumération de comptes)
Autorisation	Vérification de propriété et de rôle systématique côté service (pas seulement côté route) ; ex. modération réservée au rôle <code>ADMIN</code> via middleware <code>requireRole</code>
Fuite d'information	Middleware d'erreur centralisé : toute erreur non prévue renvoie un message générique au client, la stack trace complète n'est journalisée que côté serveur
Brute force / credential stuffing	Limitation de débit (<code>express-rate-limit</code> , 20 requêtes / 15 min) sur les routes d'authentification
CORS	Origine restreinte à l'URL du front-end configurée en variable d'environnement, plutôt qu'une politique ouverte à tout domaine
Exposition de données via Supabase	Row Level Security activée sur les sept tables PostgreSQL (l'accès applicatif passe par le rôle propriétaire via Prisma, qui n'est pas soumis à RLS ; sans policy, l'accès via la clé publique Supabase est donc refusé par défaut)
Secrets	Aucun secret commité : <code>.env</code> exclu du dépôt, seul <code>.env.example</code> (valeurs factices) est versionné

Limite connue et assumée : l'ajout d'une version de mod accepte pour l'instant une URL de fichier fournie par le créateur plutôt qu'un véritable upload (la bibliothèque `multer` est installée mais pas encore branchée). C'est un choix de périmètre pour la durée du projet, à mentionner en perspective d'évolution plutôt qu'à présenter comme un oubli.

8. Tests

La stratégie retenue privilégie des **tests d'intégration bout-en-bout** plutôt que des tests unitaires isolés avec mocks : chaque test démarre l'application réelle (`createApp()`) et l'exerce via des requêtes HTTP (Supertest) contre les vraies bases de données de développement, puis nettoie les données créées. Ce choix donne une confiance plus directe que le système fonctionne réellement de bout en bout, au prix d'une exécution un peu plus lente qu'avec des mocks.

```
it('crée un mod (Postgres) et enregistre ses métadonnées jeu (Mongo)', async () => {
  const res = await request(app).post('/api/mods')
    .set('Authorization', `Bearer ${token}`)
    .send({ title: '... ', gameKey: 'arma-reforger', gameMetadata: { workshopId: 'RMM-001' } });

  expect(res.status).toBe(201);
  const metadata = await ModMetadataModel.findOne({ modId: res.body.id }).lean();
  expect(metadata?.fields.workshopId).toBe('RMM-001');
});
```

Suite	Couverture
tests/auth.test.ts	Inscription (validation mot de passe, unicité email), connexion (identifiants valides/invalides)
tests/mods.test.ts	Création de mod avec métadonnées croisées SQL/NoSQL, ajout de version, contrôle de propriété (rejet 403), lecture d'une fiche mod fusionnant les deux sources

Résultat : 9 tests, 9 succès, exécutés contre les environnements réels (Supabase + MongoDB).

9. Bilan et perspectives

9.1 Difficultés rencontrées

- **Absence de Docker fonctionnel** sur la machine de développement (pas de distribution WSL2 installée) : résolu en basculant PostgreSQL vers un hébergement Supabase gratuit et en découvrant qu'une instance MongoDB locale était déjà disponible en service Windows.
- **Changement d'API dans Prisma 7** : la suppression de l'URL de connexion directement dans le schéma (remplacée par un adaptateur de driver + fichier de configuration séparé) a nécessité une vérification directe du comportement de l'outil plutôt qu'une hypothèse basée sur les versions précédentes.
- **Typage strict d'Express 5** sur les paramètres de route (`string` | `string[]`) : traité par une fonction utilitaire de validation plutôt qu'un contournement de type.

9.2 Compétences mobilisées

[À personnaliser : lister les compétences que le candidat estime avoir le plus développées ou consolidées sur ce projet, en lien avec son parcours]

9.3 Perspectives d'évolution

- Upload de fichiers réel pour les versions de mods (stockage objet)
- Notifications en temps réel (WebSocket) pour le fil d'activité plutôt qu'un chargement à la demande
- Système de tags libres en complément des catégories fixes
- Tests de bout en bout côté interface (Playwright) en complément des tests d'intégration API

10. Annexes

10.1 Lexique

Terme	Définition
ORM	Object-Relational Mapping — bibliothèque faisant correspondre les tables d'une base relationnelle à des objets manipulables dans le code (ici, Prisma)
JWT	JSON Web Token — jeton signé transmettant l'identité et le rôle d'un utilisateur authentifié sans nécessiter de session stockée côté serveur
EAV	Entity-Attribute-Value — anti-pattern relationnel consistant à stocker des attributs dynamiques ligne par ligne plutôt qu'en colonnes, évité ici grâce au NoSQL
RLS	Row Level Security — mécanisme PostgreSQL restreignant l'accès aux lignes d'une table selon le rôle de connexion

10.2 Sources et ressources

- Dépôt du projet : github.com/Maximus04200/ModHub
- Documentation officielle Prisma, Express, React, MongoDB/Mongoose